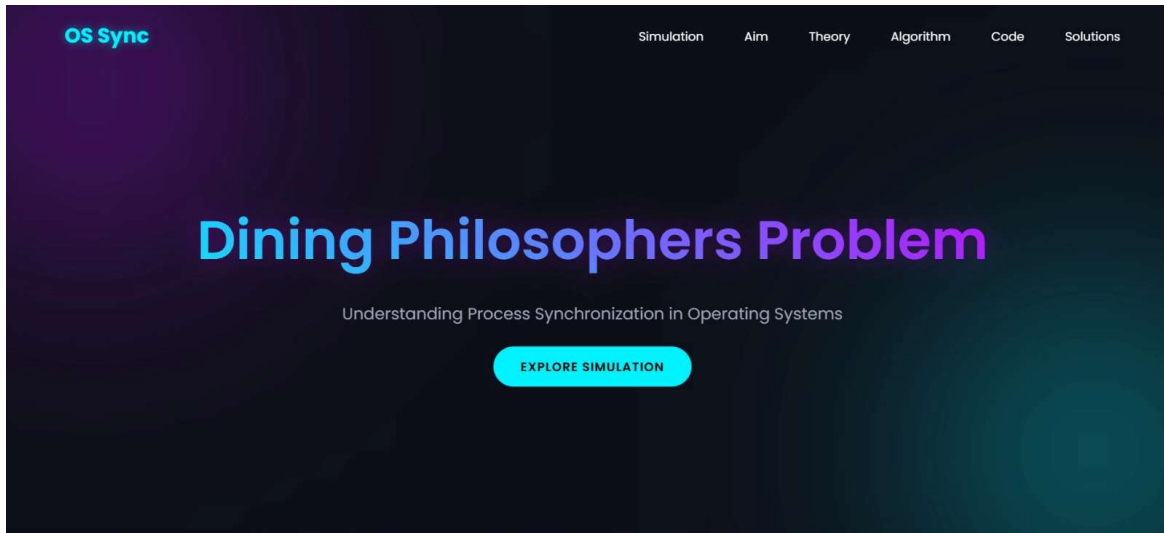


POSTLAB OUTPUT

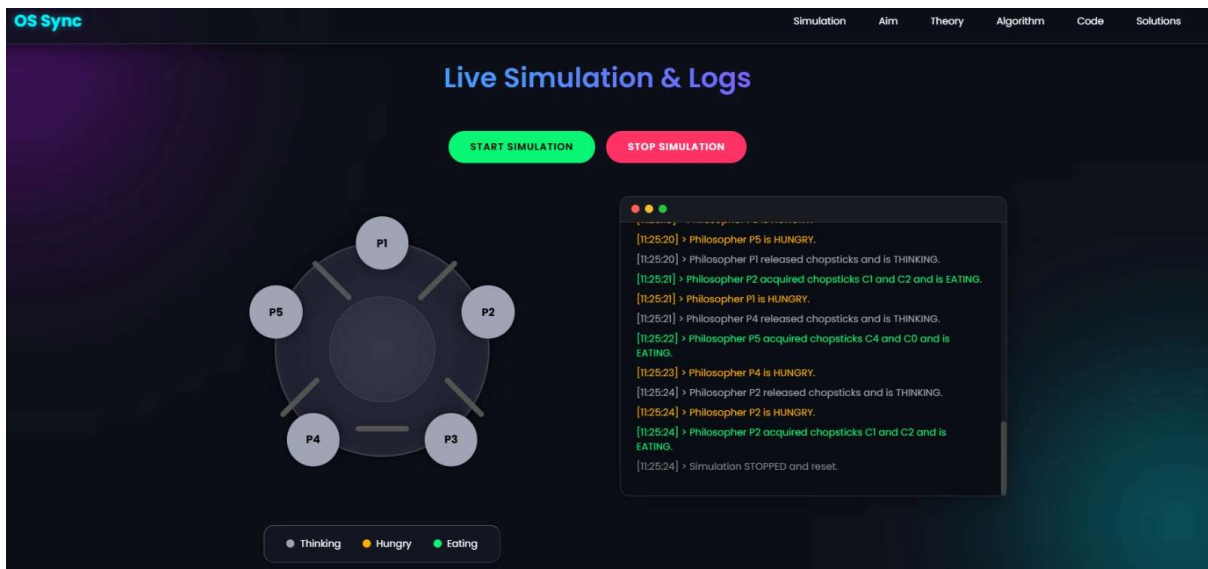
1. Dining Philosophers Problem

An interactive web-based simulation to understand process synchronization in operating systems through real-time visualization.



2. Live Simulation & Logs

Displays real-time state changes of philosophers (thinking, hungry, eating) along with logs to track resource allocation and execution flow.

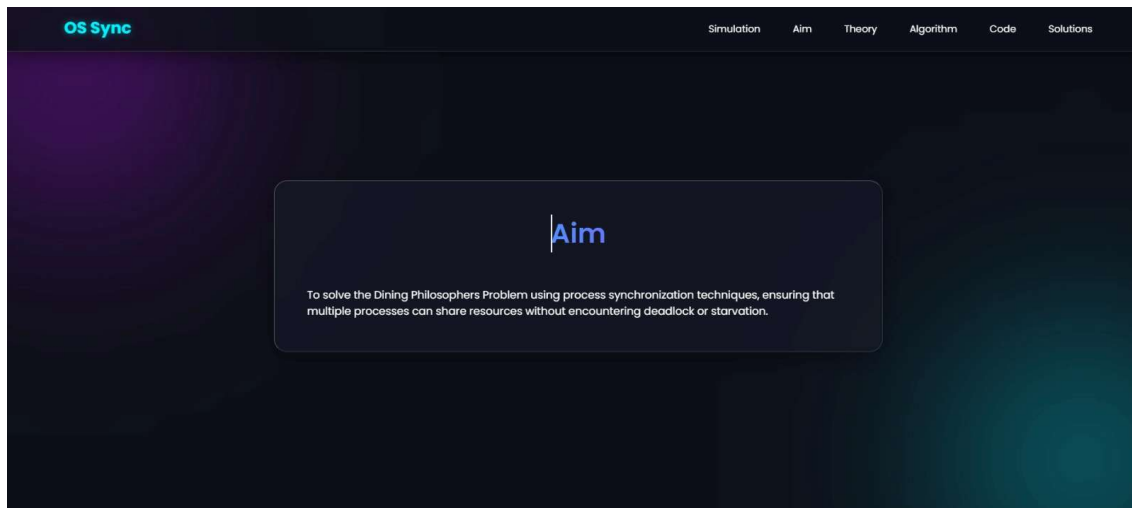


Website Link

Project URL: <https://ayush21-r.github.io/os-dining-philosophers-simulation/>

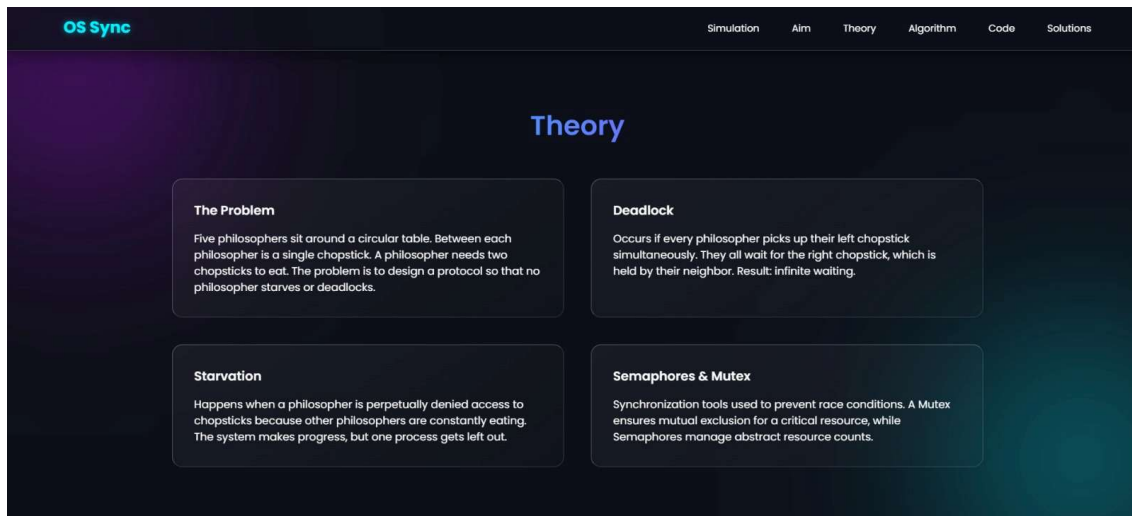
Aim

To solve the Dining Philosophers Problem using process synchronization techniques, ensuring that multiple processes share resources without deadlock or starvation.



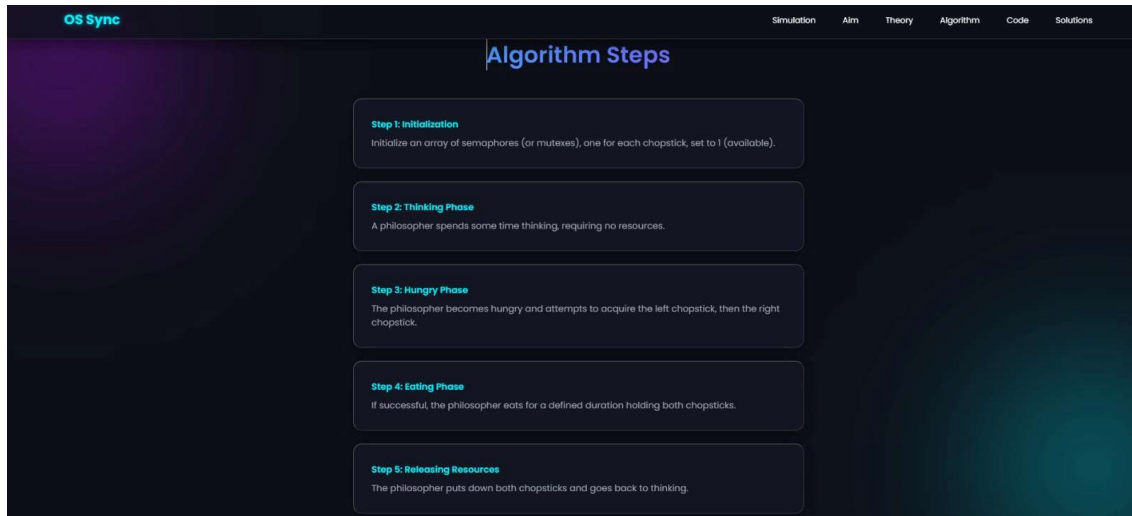
Theory

Explains the problem setup and key concepts like deadlock, starvation, mutual exclusion, and the role of semaphores and mutex in synchronization.



Algorithm Steps

Provides a step-by-step approach showing how philosophers think, become hungry, acquire chopsticks, eat, and release resources safely.



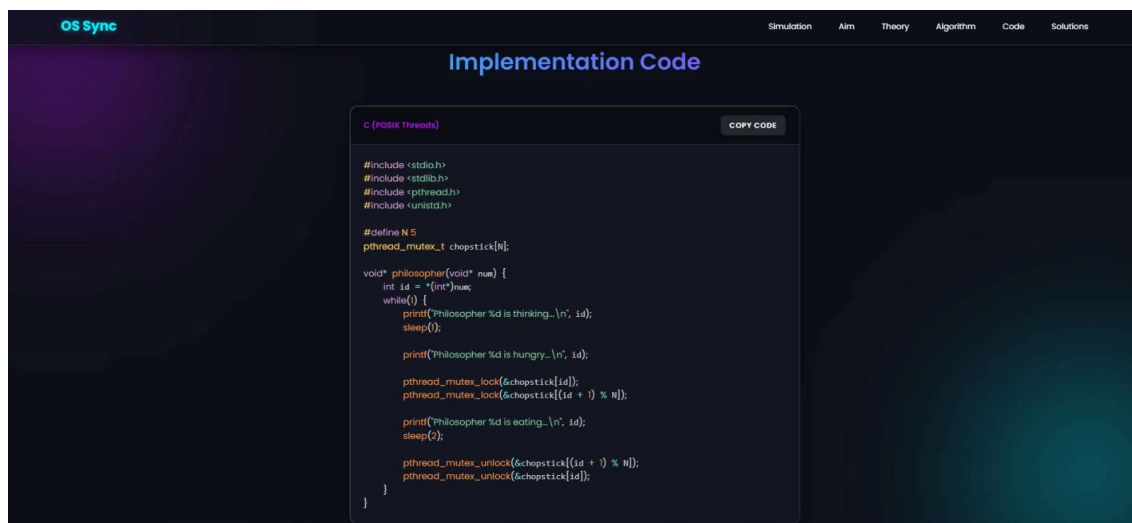
OS Sync Simulation Aim Theory Algorithm Code Solutions

Algorithm Steps

- Step 1: Initialization**
Initialize an array of semaphores (or mutexes), one for each chopstick, set to 1 (available).
- Step 2: Thinking Phase**
A philosopher spends some time thinking, requiring no resources.
- Step 3: Hungry Phase**
The philosopher becomes hungry and attempts to acquire the left chopstick, then the right chopstick.
- Step 4: Eating Phase**
If successful, the philosopher eats for a defined duration holding both chopsticks.
- Step 5: Releasing Resources**
The philosopher puts down both chopsticks and goes back to thinking.

Implementation Code

Includes the thread-based implementation using mutex locks to control access to shared resources (chopsticks).



OS Sync Simulation Aim Theory Algorithm Code Solutions

Implementation Code

C (POSIX Threads) [COPY CODE](#)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define N 5
pthread_mutex_t chopstick[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    while(1) {
        printf("Philosopher %d is thinking...\n", id);
        sleep(1);

        printf("Philosopher %d is hungry...\n", id);

        pthread_mutex_lock(&chopstick[id]);
        pthread_mutex_lock(&chopstick[(id + 1) % N]);

        printf("Philosopher %d is eating...\n", id);
        sleep(2);

        pthread_mutex_unlock(&chopstick[(id + 1) % N]);
        pthread_mutex_unlock(&chopstick[id]);
    }
}
```

Conclusion

Demonstrates how proper synchronization techniques prevent deadlock and ensure efficient and fair resource utilization. The working model is available at: <https://ayush21-r.github.io/os-dining-philosophers-simulation/>

